

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #7 - Semana 8 – Libro C - Capítulo 3
Flujo de control

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 16/09/2024

Bibliografía:

B. W. Kernighan and D. M. Ritchie, *The C programming language*, 2nd ed. Pearson, 2002, pp. 52–60.

Abstract—*This paper discusses control flow in programming, focusing on statements, blocks, and conditional structures such as if-else, else-if, and switch in C. It explains how these constructs manage execution order and decision-making. The paper also covers loops (while, for, do-while) and control flow modifiers (break, continue), illustrated with example code for character counting and string manipulation.*

I. Introducción

El control del flujo en un lenguaje de programación define el orden en el que se ejecutan las operaciones. El flujo de control establece cómo se toman decisiones y qué acciones se ejecutan, dependiendo de condiciones y reglas definidas dentro de un programa. En este resumen, se cubrirán los conceptos de sentencias y bloques, estructuras condicionales como if-else y switch, y se explicará su importancia en la ejecución controlada de programas.

II. Sentencias y bloques

A. Sentencias

En C, una sentencia es una instrucción que termina con un punto y coma (;). Ejemplos incluyen:

```
x = 0;  
i++;  
printf(...);
```

El punto y coma en C es un terminador de sentencias, a diferencia de otros lenguajes como Pascal, donde es un separador. Cada sentencia realiza una operación en particular, como asignar valores o llamar funciones.

B. Bloques

Un bloque agrupa varias sentencias entre llaves {}. Esto convierte a varias sentencias en una unidad lógica, tratada como una sola sentencia. Los bloques son útiles en estructuras como if, else, while, o for, permitiendo ejecutar múltiples operaciones como una sola.

Ejemplo de un bloque:

```
{
    x = 0;
    i++;
    printf("Hola");
}
```

No es necesario un punto y coma después de las llaves que cierran un bloque.

III. Estructura If-Else

La estructura if-else permite tomar decisiones dentro de un programa. Su sintaxis es la siguiente:

```
if (expresión)
    sentencia1
else
    sentencia2
```

A. Evaluación

Si la expresión es verdadera (valor diferente de cero), se ejecuta sentencia1. Si es falsa (valor cero) y existe un bloque else, se ejecuta sentencia2.

B. Asociación de Else

Una peculiaridad del else es que siempre se asocia con el if más cercano sin else, lo que puede llevar a ambigüedades. Para evitar errores, es recomendable usar llaves para aclarar la asociación.

Ejemplo:

```
if (n > 0) {
    if (a > b)
        z = a;
} else
    z = b;
```

Sin llaves, el else se asociaría con el if más cercano, creando confusión.

IV. Else-If

El uso de múltiples else if permite manejar varias condiciones de manera secuencial. La sintaxis es la siguiente:

```
if (expresión1)
    sentencia1
else if (expresión2)
    sentencia2
else
```

```
sentencia3
```

A. Evaluación Secuencial

Las expresiones se evalúan en orden, y cuando una es verdadera, se ejecuta la sentencia correspondiente y se detiene la evaluación de las demás. Si ninguna es verdadera, se ejecuta la última sentencia asociada al else.

Este enfoque es útil para realizar decisiones de múltiples vías.

V. Switch

El switch es una estructura que facilita la selección entre múltiples casos basados en el valor de una expresión. La sintaxis básica es:

```
switch (expresión) {
    case valor1:
        sentencia;
        break;
    case valor2:
        sentencia;
        break;
    default:
        sentencia;
}
```

A. Casos y Break

Cada caso evalúa si la expresión coincide con un valor constante. Si coincide, se ejecuta la sentencia asociada. El uso de break es fundamental para evitar que la ejecución continúe en los siguientes casos. Si no hay coincidencia, se ejecuta el bloque default si está presente.

B. Ejemplo de Uso

Un programa para contar dígitos, espacios y otros caracteres podría usar un switch para manejar cada tipo de carácter:

```
switch (c) {
    case '0': case '1': case '2':
        ndigit[c-'0']++;
        break;
    case ' ':
        nwhite++;
        break;
    default:
        nother++;
        break;
}
```

El break asegura que el flujo salga del switch al finalizar cada caso.

VI. Ejemplo completo de código

A continuación, se muestra un ejemplo de código que combina todos los conceptos explicados: sentencias, bloques, if-else, else-if y switch.

```
#include <stdio.h>
```

```
// Función para contar caracteres, dígitos, espacios y otros
void contar_caracteres() {
    int c, i, nwhite, nother, ndigit[10];
    nwhite = nother = 0;

    // Inicializar el contador de dígitos
    for (i = 0; i < 10; i++)
        ndigit[i] = 0;

    // Leer caracteres hasta EOF
    while ((c = getchar()) != EOF) {
        if (c >= '0' && c <= '9') {
            // Contar dígitos
            ndigit[c - '0']++;
        } else if (c == ' ' || c == '\n' || c == '\t') {
            // Contar espacios
            nwhite++;
        } else {
            // Contar otros caracteres
            nother++;
        }
    }

    // Imprimir resultados
    printf("Dígitos = ");
    for (i = 0; i < 10; i++)
        printf(" %d", ndigit[i]);
    printf(", Espacios en blanco = %d, Otros = %d\n", nwhite, nother);
}

// Función principal
int main() {
    contar_caracteres();
    return 0;
}
```

A. Explicación del Código

- **Sentencias y Bloques:** El código usa sentencias simples para contar dígitos, espacios y otros caracteres, agrupadas dentro de bloques {}.
- **If-Else:** Evalúa si el carácter ingresado es un dígito, un espacio o un carácter distinto usando if-else.
- **Else-If:** El flujo condicional se controla usando múltiples else if para manejar diferentes tipos de caracteres.
- **Switch:** No se utiliza directamente en este ejemplo, pero el control condicional que presenta if-else puede ser reescrito usando switch.

Este código ilustra cómo se aplican los conceptos de flujo de control en C para resolver un problema práctico.

VII. Introducción a los Bucles

En C, los bucles permiten ejecutar un conjunto de instrucciones repetidamente. Existen tres tipos principales de bucles: while, for, y do-while. Estos permiten realizar iteraciones hasta que una condición específica deje de cumplirse.

VIII. Bucle while

El bucle while evalúa una expresión al inicio de cada iteración. Si la expresión es verdadera (no cero), se ejecuta el bloque de código y luego se reevalúa la expresión. El ciclo continúa hasta que la expresión se evalúa como falsa (cero).

Sintaxis:

```
while (expresión)
    instrucción;
```

El bucle finaliza cuando la expresión se evalúa como falsa.

IX. Bucle for

El bucle for es una estructura más compacta para controlar las iteraciones. Tiene tres partes: inicialización, condición y actualización, que se ejecutan en un ciclo continuo.

Sintaxis:

```
for (inicialización; condición; actualización)
    instrucción;
```

El flujo de control del for es equivalente a un bucle while estructurado, pero centraliza el control del ciclo.

X. Bucle do-while

A diferencia de while y for, el bucle do-while evalúa la condición al final de cada iteración. Esto asegura que el bloque de código dentro del bucle se ejecute al menos una vez.

Sintaxis:

```
do {
    instrucción;
} while (expresión);
```

El bucle ejecuta el bloque y luego evalúa la expresión. Si es verdadera, repite el ciclo.

XI. Instrucciones break y continue

1. **break:** Permite salir de un bucle antes de que la condición se cumpla, terminando inmediatamente el ciclo.
2. **continue:** Omite el resto del código dentro del bucle en la iteración actual y pasa directamente a la siguiente iteración (evaluando la condición en for o while).

XII. Uso del operador coma en bucles

El operador coma , permite evaluar múltiples expresiones en una sola línea, ejecutándolas de izquierda a derecha. Es útil en bucles for cuando se desea manejar varias variables a la vez, como se muestra en el siguiente ejemplo:

```
for (i = 0, j = strlen(s) - 1; i < j; i++, j--)
    ...
```

XIII. Ejemplo de Código

El programa convierte una cadena de caracteres que representa un número en su equivalente entero (atoi) y luego invierte una cadena (reverse), utilizando tanto bucles for como while, y manejando espacios en blanco y signos.

Código:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
// Función atoi: convierte una cadena a un número entero
int atoi(const char s[]) {
    int i, n, sign;
    // Saltar espacios en blanco
    for (i = 0; isspace(s[i]); i++);

    // Determinar el signo
    sign = (s[i] == '-') ? -1 : 1;
    if (s[i] == '+' || s[i] == '-')
        i++;

    // Convertir la parte numérica
    for (n = 0; isdigit(s[i]); i++)
        n = 10 * n + (s[i] - '0');

    return sign * n;
}
// Función para invertir una cadena
void reverse(char s[]) {
    int i, j, temp;
    for (i = 0, j = strlen(s) - 1; i < j; i++, j--) {
        // Intercambiar caracteres
        temp = s[i];
        s[i] = s[j];
        s[j] = temp;
    }
}

int main() {
    char numStr[] = " -12345";
    char text[] = "Hello World!";

    // Convertir la cadena numStr a un número entero
    int num = atoi(numStr);
    printf("Número convertido: %d\n", num);

    // Invertir la cadena text
    reverse(text);
```

```
printf("Cadena invertida: %s\n", text);  
  
return 0;  
}
```

Explicación del código:

1. atoi:

- Usa un bucle for para saltar los espacios en blanco iniciales.
- Detecta el signo del número y lo almacena.
- Convierte los caracteres numéricos en un número entero.

2. reverse:

- Utiliza un bucle for con dos índices (i, j) para invertir los caracteres de una cadena.